

Ejercicios de práctica

Fundamentos Python (3)

Funciones: definición, parámetros y return

Objetivo	Practicar la definición, la llamada y la corrección de funciones en Python, distinguiendo qué entra, qué ocurre dentro y qué sale.
Temas	def y llamada, parámetros y argumentos, valores por defecto, return, print frente a return, ámbito local y global, y funciones que reciben colecciones.
Modalidad	Ejercicios incompletos para resolver individualmente. No se incluyen soluciones en este documento.

Cómo trabajar: complete cada bloque marcado con ___, ejecute el código y compare con el resultado esperado antes de avanzar al siguiente ejercicio.

Regla de trabajo

- Los espacios marcados con ___ deben ser completados.
- En los ejercicios de predicción, escriba la salida antes de ejecutar.
- En los ejercicios de corrección, modifique solo lo necesario para conservar la intención original.
- Cuide la indentación: en Python define el cuerpo de la función.
- Antes de escribir una función, pregúntese qué recibe y qué debe devolver.

A. Definir y Llamar

La función se crea una vez y se usa muchas veces

1 Definir una función

BÁSICO

Complete la palabra clave que define una función.

```
___ saludar():  
    print("Hola")  
  
saludar()
```

Resultado esperado: Se imprime Hola.

2 Llamar a la función

BÁSICO

Complete la línea que ejecuta la función ya definida.

```
def bienvenida():  
    print("Bienvenido al curso")  
  
___
```

Resultado esperado: Se imprime Bienvenido al curso.

3 Predice el orden de ejecución

BÁSICO

Sin ejecutar, determine en qué orden aparecen los mensajes.

```
def uno():  
    print("A")  
  
print("B")  
uno()  
print("C")
```

Resultado esperado: Tres líneas en el orden en que se ejecutan, no en el que se escriben.

Respuesta breve: escriba las tres líneas de salida en orden.

4 Corrige el orden

INTERMEDIO

El programa falla porque la función se llama antes de existir. Reescriba el bloque completo en el orden correcto.

```
saludar("Ana")

def saludar(nombre):
    print("Hola,", nombre)
```

Resultado esperado: Produce NameError hasta que la definición se escriba antes de la llamada.

Tu respuesta:

5 Función con un parámetro

BÁSICO

Complete el parámetro que recibe la función.

```
def saludar(__):
    print("Hola,", nombre)

saludar("Ana")
```

Resultado esperado: Hola, Ana

6 Función con dos parámetros

BÁSICO

Complete los parámetros necesarios para calcular el área.

```
def area(__, __):
    return base * altura

print(area(3, 4))
```

Resultado esperado: 12

7

Completa la llamada

BÁSICO

Complete los argumentos para obtener exactamente el resultado indicado.

```
def area(base, altura):  
    return base * altura  
  
print(area(___, ___))    # 25.0
```

Resultado esperado: 25.0

8

Reutilizar la misma función

BÁSICO

Llame tres veces a la misma función con precios distintos.

```
def con_iva(precio):  
    return precio * 1.19  
  
print(con_iva(___))  
print(con_iva(___))  
print(con_iva(___))
```

Resultado esperado: Tres valores distintos calculados con una sola fórmula.

B. Parámetros y argumentos

Lo que la función recibe para trabajar

9 Parámetro o argumento

BÁSICO

Observe la definición y la llamada, y distinga cada uno de los dos nombres.

```
def doble(n):  
    return n * 2  
  
resultado = doble(5)
```

Resultado esperado: n es el parámetro; 5 es el argumento.

Respuesta breve: ¿cuál es el parámetro y cuál es el argumento?

10 Argumentos por posición

BÁSICO

Complete la llamada para que se imprima: Ana 20 Bogota

```
def perfil(nombre, edad, ciudad):  
    print(nombre, edad, ciudad)  
  
perfil(__, __, __)
```

Resultado esperado: Ana 20 Bogota

11 Argumentos por nombre

BÁSICO

Complete los nombres de los parámetros para que el orden deje de importar.

```
def perfil(nombre, edad, ciudad):  
    print(nombre, edad, ciudad)  
  
perfil(__=20, __="Ana", __="Bogota")
```

Resultado esperado: Ana 20 Bogota

12 Valor por defecto

BÁSICO

Complete el valor por defecto del parámetro saludo.

```
def saludar(nombre, saludo=___):  
    print(saludo, nombre)  
  
saludar("Ana")
```

Resultado esperado: Hola Ana

13 Reemplazar el valor por defecto

BÁSICO

Complete el argumento que sustituye al saludo por defecto.

```
def saludar(nombre, saludo="Hola"):  
    print(saludo, nombre)  
  
saludar("Luis", ___)
```

Resultado esperado: Buen dia Luis

14 Orden de los parámetros

INTERMEDIO

El siguiente encabezado produce un error. Reescriba la definición completa de forma correcta.

```
def registrar(cantidad=1, producto):  
    print(producto, cantidad)
```

Resultado esperado: SyntaxError: los parámetros con valor por defecto van siempre al final.

Tu respuesta:

15 Una lista como argumento

INTERMEDIO

Complete el recorrido para sumar todos los precios recibidos.

```
def total(precios):  
    suma = 0  
    for p in ____:  
        suma = suma + p  
    return suma  
  
print(total([1200, 950, 3400]))
```

Resultado esperado: 5550

C. return y resultados

Lo que la función entrega al programa

16 Devolver un valor

BÁSICO

Complete la palabra clave que entrega el resultado.

```
def doble(n):  
    ____ n * 2  
  
print(doble(5))
```

Resultado esperado: 10

17 Usar el valor devuelto

BÁSICO

Complete la llamada para guardar el resultado y operar con él.

```
def doble(n):  
    return n * 2  
  
resultado = ____ (6)  
print(resultado + 1)
```

Resultado esperado: 13

18 Función sin return

BÁSICO

Sin ejecutar, determine qué imprimen las dos últimas líneas.

```
def saludo(nombre):  
    print("Hola,", nombre)  
  
x = saludo("Ana")  
print(x)
```

Resultado esperado: Hola, Ana y luego None.

Respuesta breve: escriba las dos líneas de salida y explique la segunda.

19 print o return

INTERMEDIO

La suma falla. Corrija la función para que el resultado pueda seguir usándose y escriba la versión corregida.

```
def doble(n):  
    print(n * 2)  
  
total = doble(5) + 3
```

Resultado esperado: TypeError hasta cambiar print por return; después total vale 13.

Tu respuesta:

20 return dentro de una condición

INTERMEDIO

Complete la instrucción que devuelve el segundo resultado.

```
def signo(n):  
    if n < 0:  
        return "negativo"  
    ____ "positivo"  
  
print(signo(-4))  
print(signo(7))
```

Resultado esperado: negativo y positivo

21 return termina la función

INTERMEDIO

Sin ejecutar, determine qué se imprime y qué línea nunca se ejecuta.

```
def prueba(n):  
    if n > 0:  
        return "positivo"  
    print("línea intermedia")  
    return "otro"  
  
print(prueba(5))
```

Resultado esperado: Solo se imprime positivo.

Respuesta breve: ¿por qué no aparece la línea intermedia?

22 Devolver dos valores

INTERMEDIO

Complete la función para que entregue el mínimo y el máximo.

```
def resumen(valores):  
    return min(valores), ____(valores)  
  
menor, mayor = resumen([8, 3, 10, 5])  
print(menor, mayor)
```

Resultado esperado: 3 10

23 Encadenar funciones

INTERMEDIO

Complete la llamada interna para calcular el precio con IVA redondeado.

```
def con_iva(p):  
    return p * 1.19  
  
def redondear(valor):  
    return round(valor, 2)  
  
print(redondear(____(1200)))
```

Resultado esperado: 1428.0

D. Ámbito y colecciones

Qué vive dentro de la función y qué vive fuera

24 Variable local y global

INTERMEDIO

Sin ejecutar, determine qué imprime cada una de las dos llamadas.

```
mensaje = "global"

def prueba():
    mensaje = "local"
    print(mensaje)

prueba()
print(mensaje)
```

Resultado esperado: local y luego global.

Respuesta breve: escriba las dos líneas de salida.

25 Evitar las variables globales

INTERMEDIO

Reescriba la función para que reciba el IVA como parámetro y no dependa de una variable externa.

```
iva = 0.19

def con_iva(precio):
    return precio * (1 + iva)
```

Resultado esperado: Una función que recibe precio e iva y devuelve el resultado.

Tu respuesta:

26 No modificar el original

INTERMEDIO

Complete la función para que devuelva una lista nueva sin alterar la que recibe.

```
def agregar(lista):  
    nueva = lista + [____]  
    return nueva  
  
datos = [1, 2]  
print(agregar(datos))  
print(datos)
```

Resultado esperado: [1, 2, 3] y luego [1, 2]

27 Función que recibe una lista

INTERMEDIO

Complete la función que calcula el promedio de una lista de notas.

```
def promedio(notas):  
    total = 0  
    for n in notas:  
        total = total + n  
    return total / ____ (notas)  
  
print(promedio([3.5, 4.2, 2.8]))
```

Resultado esperado: 3.5

28 Función que recibe un diccionario

INTERMEDIO

Complete las claves para mostrar el nombre y la nota del estudiante.

```
def describir(alumno):  
    print(alumno[____], alumno[____])  
  
describir({"nombre": "Laura", "nota": 4.6})
```

Resultado esperado: Laura 4.6

29 Función que devuelve una lista

AVANZADO

Complete el método que agrega elementos y el valor que se devuelve.

```
def aprobados(estudiantes):  
    resultado = []  
    for e in estudiantes:  
        if e["nota"] >= 3.0:  
            resultado.___(e["nombre"])  
    return ____  
  
datos = [{"nombre": "Ana", "nota": 4.2},  
         {"nombre": "Luis", "nota": 2.8}]  
print(aprobados(datos))
```

Resultado esperado: ['Ana']

Reto integrador

Tres funciones que se llaman entre sí para producir un reporte

30 Reporte de notas con funciones

AVANZADO

Complete el programa que calcula el promedio de un estudiante, decide si aprueba y muestra el reporte usando tres funciones.

```
def promedio(notas):  
    total = 0  
    for n in notas:  
        total = total + n  
    return total / len(notas)  
  
def aprueba(prom, minimo=____):  
    return prom ____ minimo  
  
def reporte(nombre, notas):  
    prom = ____ (notas)  
    print(nombre, round(prom, 2))  
    if ____ (prom):  
        print("Aprobado")  
    else:  
        print("No aprobado")  
  
reporte("Laura", [3.5, 4.2, 2.8])
```

Resultado esperado: Laura 3.5 y luego Aprobado.

Respuesta breve: indique qué recibe y qué devuelve cada una de las tres funciones.

Cierre y pauta de revisión

Verificación final antes de entregar el cuaderno

Antes de entregar o compartir el cuaderno, verifique que el código se ejecuta de inicio a fin y que los resultados coinciden con los resultados esperados. Cuando un resultado no coincida, revise primero el orden de definición, los nombres de los parámetros y si la función devuelve o solo imprime.

Elemento	Uso esperado en los ejercicios
def y nombre	Declarar la función con un nombre que explique qué hace.
Parámetros	Recibir los datos que la función necesita para trabajar.
return	Entregar un resultado que el programa pueda seguir usando.
Ámbito	Distinguir lo que vive dentro de la función de lo que vive fuera.

Referencias breves

- Python Software Foundation. The Python Tutorial: Defining Functions.
- Joyanes Aguilar (2020). Fundamentos de programación.
- Ramírez Jiménez (2021). Python a Fondo.

Autoevaluación rápida

1. ¿Puedo definir una función y llamarla en el orden correcto?
2. ¿Puedo distinguir entre un parámetro y un argumento?
3. ¿Puedo explicar la diferencia entre print y return?
4. ¿Puedo decidir cuándo conviene extraer una función?